CS 4420 Assignment 05

Jacob Rowe

1. Overview

This project simulates three CPU scheduling policies: First Come First Served (FCFS), Round Robin (RR), and Shortest Job First (SJF). The program is written in C (source file proj2.c). It does not create or run real processes. It reads task information from an input file, simulates the CPU one millisecond at a time, and prints which task is chosen in every time slot. When all tasks finish, it prints start, end, running, and waiting time for each task and the average waiting time.

1.1 Design

- **Task table:** every task from the input file is stored in a static array with its pid, arrival time, burst time, remaining time, start time, and end time.
- **Ready queue:** a static array of task indices with a maximum length of 20. It supports push at the back and removal from any position.
- **Scheduler functions:** pick_fcfs(), pick_rr(), and pick_sjf() each remove one task from the ready queue and return it (or report that the queue is empty). FCFS and RR take the front of the queue. SJF takes the task with the smallest burst time.
- **Time-driven loop:** in each time slot the simulator (1) adds newly arrived tasks to the ready queue, (2) puts a task preempted in the previous slot back in the queue, (3) calls the scheduler if the CPU is free, (4) prints the chosen task, or "idle" if nothing is ready, and (5) runs the task for one time unit and updates its state. RR preempts a task when it has used its full quantum.

1.2 Conventions used

- Waiting time = end time - arrival time - burst time. For FCFS and SJF this equals start time - arrival time.
- In RR, if a new task arrives at the same instant another task is preempted, the new task enters the ready queue first and the preempted task goes behind it. This reproduces the sample output in the lab handout.
- SJF ties (equal burst time) are broken by earlier arrival time, then by the order in the input file. SJF is non-preemptive.
- The simulator accepts up to 20 tasks, matching the ready-queue limit.

2. How to Compile and Run

2.1 Compile

The program needs only a standard C compiler (gcc on Linux or macOS). From the folder that contains proj2.c run:

```
gcc -Wall -Wextra -o proj2 proj2.c
```

It compiles with no warnings and produces the executable proj2.

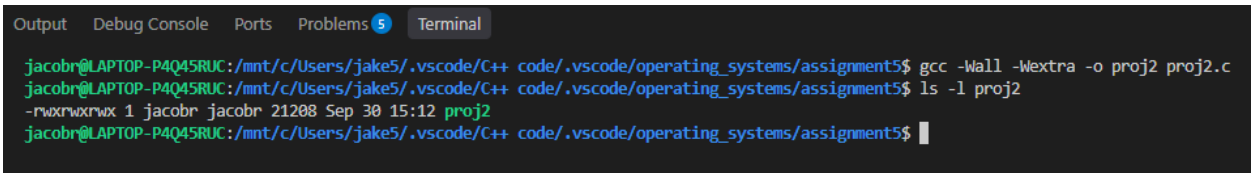


Figure 1: Compiling proj2.c with warnings enabled

2.2 Input file format

The first line holds the number of tasks. Each following line holds three integers: pid, arrival_time (ms), and burst_time (ms). Example input.1:

```
4
0 0 12
1 2 4
2 3 1
3 4 2
```

2.3 Run

```
./proj2 input_file [FCFS|RR|SJF] [time_quantum]
```

The time quantum is required for RR (even when it is 1) and is ignored by FCFS and SJF. Examples:

```
./proj2 input.1 FCFS
./proj2 input.1 RR 2
./proj2 input.1 SJF
```

For every time slot the program prints a line such as "[4- 5] run P0 rem=7 ready=[P1 P2 P3]" or "[5- 6] idle". It also prints an event line whenever a task arrives, starts, resumes, is preempted, or completes. At the end it prints the statistics tables and the average waiting time. Invalid arguments or input files produce a usage or error message and a non-zero exit status.

3. Test Cases

Four test cases were developed. Test cases 1 to 3 cover the scheduling algorithms, and test case 4 covers command-line error handling. Expected values for test case 1 come from the lab handout. Expected values for test cases 2 and 3 were computed independently with a separate Python reference implementation and checked by hand for FCFS and SJF.

TC	Input	Purpose	Commands	Expected avg. wait
1	input.1	Lab handout sample; compare with the published output	FCFS; SJF; RR 5	9; 7.75; 5.5
2	input.2	Gaps between arrivals so the CPU is idle	FCFS; RR 2	0.25; 0.25
3	input.3	Same-time arrivals, unsorted pids, SJF ties	FCFS; SJF; RR 3	6.6; 3.6; 6.2
4	(various)	Bad arguments and missing file	5 invalid commands	Error message, no crash

Test Case 1: Lab handout sample (input.1)

Confirm that all three algorithms reproduce the sample output in the lab handout, including the RR execution trace and the order in which SJF runs tasks.

pid	arrival_time	burst_time
0	0	12
1	2	4
2	3	1
3	4	2

Expected per-task waiting times from the handout: FCFS = 0, 10, 13, 13 (average 9). SJF = 0, 9, 9, 13 for pids 0, 2, 3, 1 (average 7.75). RR with quantum 5 = 7, 3, 6, 6 (average 5.5). The program output below matches all of these values, and the RR trace lists the same six CPU turns as the handout (P0, P1, P2, P3, P0, P0).

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 FCFS
Loaded 4 task(s) from input.1.  Algorithm: FCFS

        t=0    P0 arrives (burst 12)
        t=0    P0 starts running
[  0-  1] run P0   rem=11    ready=[ ]
[  1-  2] run P0   rem=10    ready=[ ]
        t=2    P1 arrives (burst 4)
[  2-  3] run P0   rem=9     ready=[P1]
        t=3    P2 arrives (burst 1)
[  3-  4] run P0   rem=8     ready=[P1 P2]
        t=4    P3 arrives (burst 2)
[  4-  5] run P0   rem=7     ready=[P1 P2 P3]
[  5-  6] run P0   rem=6     ready=[P1 P2 P3]
[  6-  7] run P0   rem=5     ready=[P1 P2 P3]
[  7-  8] run P0   rem=4     ready=[P1 P2 P3]
[  8-  9] run P0   rem=3     ready=[P1 P2 P3]
[  9- 10] run P0   rem=2     ready=[P1 P2 P3]
[ 10- 11] run P0   rem=1     ready=[P1 P2 P3]
[ 11- 12] run P0   rem=0     ready=[P1 P2 P3]
        t=12   P0 completes
        t=12   P1 starts running
[ 12- 13] run P1   rem=3     ready=[P2 P3]
[ 13- 14] run P1   rem=2     ready=[P2 P3]
[ 14- 15] run P1   rem=1     ready=[P2 P3]
[ 15- 16] run P1   rem=0     ready=[P2 P3]
        t=16   P1 completes
        t=16   P2 starts running
[ 16- 17] run P2   rem=0     ready=[P3]
        t=17   P2 completes
        t=17   P3 starts running
[ 17- 18] run P3   rem=1     ready=[ ]
[ 18- 19] run P3   rem=0     ready=[ ]
        t=19   P3 completes

FCFS:

PID    Arrival Time  Start Time  End Time  Running Time  Waiting Time
0      0             0           12        12            0
1      2             12          16        4             10
2      3             16          17        1             13
3      4             17          19        2             13

Average Waiting Time: 9
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 2: Test case 1, ./proj2 input.1 FCFS (average waiting time 9)

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 SJF
Loaded 4 task(s) from input.1.  Algorithm: SJF

        t=0    P0 arrives (burst 12)
        t=0    P0 starts running
[  0-  1] run P0   rem=11    ready=[ ]
[  1-  2] run P0   rem=10    ready=[ ]
        t=2    P1 arrives (burst 4)
[  2-  3] run P0   rem=9     ready=[P1]
        t=3    P2 arrives (burst 1)
[  3-  4] run P0   rem=8     ready=[P1 P2]
        t=4    P3 arrives (burst 2)
[  4-  5] run P0   rem=7     ready=[P1 P2 P3]
[  5-  6] run P0   rem=6     ready=[P1 P2 P3]
[  6-  7] run P0   rem=5     ready=[P1 P2 P3]
[  7-  8] run P0   rem=4     ready=[P1 P2 P3]
[  8-  9] run P0   rem=3     ready=[P1 P2 P3]
[  9- 10] run P0   rem=2     ready=[P1 P2 P3]
[ 10- 11] run P0   rem=1     ready=[P1 P2 P3]
[ 11- 12] run P0   rem=0     ready=[P1 P2 P3]
        t=12   P0 completes
        t=12   P2 starts running
[ 12- 13] run P2   rem=0     ready=[P1 P3]
        t=13   P2 completes
        t=13   P3 starts running
[ 13- 14] run P3   rem=1     ready=[P1]
[ 14- 15] run P3   rem=0     ready=[P1]
        t=15   P3 completes
        t=15   P1 starts running
[ 15- 16] run P1   rem=3     ready=[ ]
[ 16- 17] run P1   rem=2     ready=[ ]
[ 17- 18] run P1   rem=1     ready=[ ]
[ 18- 19] run P1   rem=0     ready=[ ]
        t=19   P1 completes

SJF:

PID    Arrival Time  Start Time  End Time  Running Time  Waiting Time
0      0             0           12        12            0
2      3             12          13        1             9
3      4             13          15        2             9
1      2             15          19        4             13

Average Waiting Time: 7.75
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 3: Test case 1, ./proj2 input.1 SJF (average waiting time 7.75)

```
Output   Debug Console   Ports   Problems 5   Terminal

jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 RR 5
Loaded 4 task(s) from input.1.  Algorithm: RR (quantum 5)

        t=0    P0 arrives (burst 12)
        t=0    P0 starts running
[  0-  1] run P0   rem=11    ready=[ ]
[  1-  2] run P0   rem=10    ready=[ ]
        t=2    P1 arrives (burst 4)
[  2-  3] run P0   rem=9     ready=[P1]
        t=3    P2 arrives (burst 1)
[  3-  4] run P0   rem=8     ready=[P1 P2]
        t=4    P3 arrives (burst 2)
[  4-  5] run P0   rem=7     ready=[P1 P2 P3]
        t=5    P0 quantum expired, preempted
        t=5    P1 starts running
[  5-  6] run P1   rem=3     ready=[P2 P3 P0]
[  6-  7] run P1   rem=2     ready=[P2 P3 P0]
[  7-  8] run P1   rem=1     ready=[P2 P3 P0]
[  8-  9] run P1   rem=0     ready=[P2 P3 P0]
        t=9    P1 completes
        t=9    P2 starts running
[  9- 10] run P2   rem=0     ready=[P3 P0]
        t=10   P2 completes
        t=10   P3 starts running
[ 10- 11] run P3   rem=1     ready=[P0]
[ 11- 12] run P3   rem=0     ready=[P0]
        t=12   P3 completes
        t=12   P0 resumes running
[ 12- 13] run P0   rem=6     ready=[ ]
[ 13- 14] run P0   rem=5     ready=[ ]
[ 14- 15] run P0   rem=4     ready=[ ]
[ 15- 16] run P0   rem=3     ready=[ ]
[ 16- 17] run P0   rem=2     ready=[ ]
        t=17   P0 quantum expired, preempted
        t=17   P0 resumes running
[ 17- 18] run P0   rem=1     ready=[ ]
[ 18- 19] run P0   rem=0     ready=[ ]
        t=19   P0 completes

RR (Time quantum = 5):

PID    Start Time  End Time  Running Time
0      0           5         5
1      5           9         4
2      9           10        1
3      10          12        2
0      12          17        5
0      17          19        2

PID    Arrival Time  Running Time  End Time  Waiting Time
0      0             12           19        7
1      2             4            9         3
2      3             1            10        6
3      4             2            12        6

Average Waiting Time: 5.5
```

Figure 4: Test case 1, ./proj2 input.1 RR 5 (average waiting time 5.5)

Test Case 2: CPU idle periods (input.2)

Check that the simulator prints "idle" when the ready queue is empty and keeps advancing time until the next task arrives. Tasks arrive far apart, so the CPU is idle at times 0-2, 5-8, 11-20, and 24-25 in the FCFS run.

pid	arrival_time	burst_time
0	2	3
1	8	2
2	9	1
3	20	4

Expected result: only P2 waits (1 ms, because P1 is still running when it arrives at time 9), so the average waiting time is 1 / 4 = 0.25. With quantum 2 no task is ever preempted in a way that changes the waiting times, so RR gives the same average. Both runs print 0.25.

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.2 FCFS
Loaded 4 task(s) from input.2.  Algorithm: FCFS

[  0-  1] idle              ready=[ ]
[  1-  2] idle              ready=[ ]
        t=2    P0 arrives (burst 3)
        t=2    P0 starts running
[  2-  3] run P0   rem=2    ready=[ ]
[  3-  4] run P0   rem=1    ready=[ ]
[  4-  5] run P0   rem=0    ready=[ ]
        t=5    P0 completes
[  5-  6] idle              ready=[ ]
[  6-  7] idle              ready=[ ]
[  7-  8] idle              ready=[ ]
        t=8    P1 arrives (burst 2)
        t=8    P1 starts running
[  8-  9] run P1   rem=1    ready=[ ]
        t=9    P2 arrives (burst 1)
[  9- 10] run P1   rem=0    ready=[P2]
        t=10   P1 completes
        t=10   P2 starts running
[ 10- 11] run P2   rem=0    ready=[ ]
        t=11   P2 completes
[ 11- 12] idle              ready=[ ]
[ 12- 13] idle              ready=[ ]
[ 13- 14] idle              ready=[ ]
[ 14- 15] idle              ready=[ ]
[ 15- 16] idle              ready=[ ]
[ 16- 17] idle              ready=[ ]
[ 17- 18] idle              ready=[ ]
[ 18- 19] idle              ready=[ ]
[ 19- 20] idle              ready=[ ]
        t=20   P3 arrives (burst 4)
        t=20   P3 starts running
[ 20- 21] run P3   rem=3    ready=[ ]
[ 21- 22] run P3   rem=2    ready=[ ]
[ 22- 23] run P3   rem=1    ready=[ ]
[ 23- 24] run P3   rem=0    ready=[ ]
        t=24   P3 completes

FCFS:

PID    Arrival Time  Start Time  End Time  Running Time  Waiting Time
0      2             2           5         3             0
1      8             8           10        2             0
2      9             10          11        1             1
3      20            20          24        4             0

Average Waiting Time: 0.25
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 5: Test case 2, ./proj2 input.2 FCFS (average waiting time 0.25)

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.2 RR 2
Loaded 4 task(s) from input.2.  Algorithm: RR (quantum 2)

[  0-  1] idle              ready=[ ]
[  1-  2] idle              ready=[ ]
        t=2    P0 arrives (burst 3)
        t=2    P0 starts running
[  2-  3] run P0   rem=2    ready=[ ]
[  3-  4] run P0   rem=1    ready=[ ]
        t=4    P0 quantum expired, preempted
        t=4    P0 resumes running
[  4-  5] run P0   rem=0    ready=[ ]
        t=5    P0 completes
[  5-  6] idle              ready=[ ]
[  6-  7] idle              ready=[ ]
[  7-  8] idle              ready=[ ]
        t=8    P1 arrives (burst 2)
        t=8    P1 starts running
[  8-  9] run P1   rem=1    ready=[ ]
        t=9    P2 arrives (burst 1)
[  9- 10] run P1   rem=0    ready=[P2]
        t=10   P1 completes
        t=10   P2 starts running
[ 10- 11] run P2   rem=0    ready=[ ]
        t=11   P2 completes
[ 11- 12] idle              ready=[ ]
[ 12- 13] idle              ready=[ ]
[ 13- 14] idle              ready=[ ]
[ 14- 15] idle              ready=[ ]
[ 15- 16] idle              ready=[ ]
[ 16- 17] idle              ready=[ ]
[ 17- 18] idle              ready=[ ]
[ 18- 19] idle              ready=[ ]
[ 19- 20] idle              ready=[ ]
        t=20   P3 arrives (burst 4)
        t=20   P3 starts running
[ 20- 21] run P3   rem=3    ready=[ ]
[ 21- 22] run P3   rem=2    ready=[ ]
        t=22   P3 quantum expired, preempted
        t=22   P3 resumes running
[ 22- 23] run P3   rem=1    ready=[ ]
[ 23- 24] run P3   rem=0    ready=[ ]
        t=24   P3 completes

RR (Time quantum = 2):

PID    Start Time  End Time  Running Time
0      2           4         2
0      4           5         1
1      8           10        2
2      10          11        1
3      20          22        2
3      22          24        2

PID    Arrival Time  Running Time  End Time  Waiting Time
0      2             3            5         0
1      8             2            10        0
2      9             1            11        1
3      20            4            24        0

Average Waiting Time: 0.25
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 6: Test case 2, ./proj2 input.2 RR 2 (average waiting time 0.25)

Test Case 3: Simultaneous arrivals, unsorted pids, SJF ties (input.3)

Stress the ordering rules. Three tasks arrive at time 0, the pids are not in increasing order, and two tasks have the same burst time (3 ms).

pid	arrival_time	burst_time
4	0	6
2	0	3
7	0	3
1	2	1
9	5	4

Expected results (hand-checked):

- FCFS runs P4, P2, P7, P1, P9 with waiting times 0, 6, 9, 10, 8 for a total of 33, so the average is 33 / 5 = 6.6.
- SJF runs P2 first (tie with P7 broken by input order), then P1, P7, P9, P4 with waiting times 0, 1, 4, 2, 11 for a total of 18, so the average is 18 / 5 = 3.6.
- RR with quantum 3 gives an average of 6.2 (from the independent reference implementation).

All three program outputs match these values. SJF shows a clear improvement over FCFS because the 6 ms task P4 runs last.

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.3 FCFS
Loaded 5 task(s) from input.3.  Algorithm: FCFS

       t=0    P4 arrives (burst 6)
       t=0    P2 arrives (burst 3)
       t=0    P7 arrives (burst 3)
       t=0    P4 starts running
[  0-  1] run P4   rem=5     ready=[P2 P7]
[  1-  2] run P4   rem=4     ready=[P2 P7]
       t=2    P1 arrives (burst 1)
[  2-  3] run P4   rem=3     ready=[P2 P7 P1]
[  3-  4] run P4   rem=2     ready=[P2 P7 P1]
[  4-  5] run P4   rem=1     ready=[P2 P7 P1]
       t=5    P9 arrives (burst 4)
[  5-  6] run P4   rem=0     ready=[P2 P7 P1 P9]
       t=6    P4 completes
       t=6    P2 starts running
[  6-  7] run P2   rem=2     ready=[P7 P1 P9]
[  7-  8] run P2   rem=1     ready=[P7 P1 P9]
[  8-  9] run P2   rem=0     ready=[P7 P1 P9]
       t=9    P2 completes
       t=9    P7 starts running
[  9- 10] run P7   rem=2     ready=[P1 P9]
[ 10- 11] run P7   rem=1     ready=[P1 P9]
[ 11- 12] run P7   rem=0     ready=[P1 P9]
       t=12   P7 completes
       t=12   P1 starts running
[ 12- 13] run P1   rem=0     ready=[P9]
       t=13   P1 completes
       t=13   P9 starts running
[ 13- 14] run P9   rem=3     ready=[ ]
[ 14- 15] run P9   rem=2     ready=[ ]
[ 15- 16] run P9   rem=1     ready=[ ]
[ 16- 17] run P9   rem=0     ready=[ ]
       t=17   P9 completes

FCFS:

PID    Arrival Time  Start Time  End Time  Running Time  Waiting Time
4      0             0           6         6             0
2      0             6           9         3             6
7      0             9           12        3             9
1      2             12          13        1             10
9      5             13          17        4             8

Average Waiting Time: 6.6
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 7: Test case 3, ./proj2 input.3 FCFS (average waiting time 6.6)

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.3 SJF
Loaded 5 task(s) from input.3.  Algorithm: SJF

       t=0    P4 arrives (burst 6)
       t=0    P2 arrives (burst 3)
       t=0    P7 arrives (burst 3)
       t=0    P2 starts running
[  0-  1] run P2   rem=2     ready=[P4 P7]
[  1-  2] run P2   rem=1     ready=[P4 P7]
       t=2    P1 arrives (burst 1)
[  2-  3] run P2   rem=0     ready=[P4 P7 P1]
       t=3    P2 completes
       t=3    P1 starts running
[  3-  4] run P1   rem=0     ready=[P4 P7]
       t=4    P1 completes
       t=4    P7 starts running
[  4-  5] run P7   rem=2     ready=[P4]
       t=5    P9 arrives (burst 4)
[  5-  6] run P7   rem=1     ready=[P4 P9]
[  6-  7] run P7   rem=0     ready=[P4 P9]
       t=7    P7 completes
       t=7    P9 starts running
[  7-  8] run P9   rem=3     ready=[P4]
[  8-  9] run P9   rem=2     ready=[P4]
[  9- 10] run P9   rem=1     ready=[P4]
[ 10- 11] run P9   rem=0     ready=[P4]
       t=11   P9 completes
       t=11   P4 starts running
[ 11- 12] run P4   rem=5     ready=[ ]
[ 12- 13] run P4   rem=4     ready=[ ]
[ 13- 14] run P4   rem=3     ready=[ ]
[ 14- 15] run P4   rem=2     ready=[ ]
[ 15- 16] run P4   rem=1     ready=[ ]
[ 16- 17] run P4   rem=0     ready=[ ]
       t=17   P4 completes

SJF:

PID    Arrival Time  Start Time  End Time  Running Time  Waiting Time
2      0             0           3         3             0
1      2             3           4         1             1
7      0             4           7         3             4
9      5             7           11        4             2
4      0             11          17        6             11

Average Waiting Time: 3.6
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 8: Test case 3, ./proj2 input.3 SJF (average waiting time 3.6)

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.3 RR 3
Loaded 5 task(s) from input.3.  Algorithm: RR (quantum 3)

        t=0    P4 arrives (burst 6)
        t=0    P2 arrives (burst 3)
        t=0    P7 arrives (burst 3)
        t=0    P4 starts running
[  0-  1] run P4   rem=5     ready=[P2 P7]
[  1-  2] run P4   rem=4     ready=[P2 P7]
        t=2    P1 arrives (burst 1)
[  2-  3] run P4   rem=3     ready=[P2 P7 P1]
        t=3    P4 quantum expired, preempted
        t=3    P2 starts running
[  3-  4] run P2   rem=2     ready=[P7 P1 P4]
[  4-  5] run P2   rem=1     ready=[P7 P1 P4]
        t=5    P9 arrives (burst 4)
[  5-  6] run P2   rem=0     ready=[P7 P1 P4 P9]
        t=6    P2 completes
        t=6    P7 starts running
[  6-  7] run P7   rem=2     ready=[P1 P4 P9]
[  7-  8] run P7   rem=1     ready=[P1 P4 P9]
[  8-  9] run P7   rem=0     ready=[P1 P4 P9]
        t=9    P7 completes
        t=9    P1 starts running
[  9- 10] run P1   rem=0     ready=[P4 P9]
        t=10   P1 completes
        t=10   P4 resumes running
[ 10- 11] run P4   rem=2     ready=[P9]
[ 11- 12] run P4   rem=1     ready=[P9]
[ 12- 13] run P4   rem=0     ready=[P9]
        t=13   P4 completes
        t=13   P9 starts running
[ 13- 14] run P9   rem=3     ready=[ ]
[ 14- 15] run P9   rem=2     ready=[ ]
[ 15- 16] run P9   rem=1     ready=[ ]
        t=16   P9 quantum expired, preempted
        t=16   P9 resumes running
[ 16- 17] run P9   rem=0     ready=[ ]
        t=17   P9 completes

RR (Time quantum = 3):

PID    Start Time  End Time  Running Time
4      0           3         3
2      3           6         3
7      6           9         3
1      9           10        1
4      10          13        3
9      13          16        3
9      16          17        1

PID    Arrival Time  Running Time  End Time  Waiting Time
4      0             6            13        7
2      0             3            6         3
7      0             3            9         6
1      2             1            10        7
9      5             4            17        8

Average Waiting Time: 6.2
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 9: Test case 3, ./proj2 input.3 RR 3 (average waiting time 6.2)

Test Case 4: Command-line error handling

Confirm that bad input is rejected with a clear message instead of a crash. The five commands are: no algorithm given, RR without a quantum, RR with quantum 0, an unknown algorithm name, and a missing input file.

```
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1
Usage: ./proj2 input_file [FCFS|RR|SJF] [time_quantum]
  time_quantum (positive integer) is required for RR and is
  ignored by FCFS and SJF.
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 RR
Error: RR requires a time quantum
Usage: ./proj2 input_file [FCFS|RR|SJF] [time_quantum]
  time_quantum (positive integer) is required for RR and is
  ignored by FCFS and SJF.
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 RR 0
Error: time quantum must be a positive integer
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 input.1 XYZ
Error: unknown algorithm 'XYZ'
Usage: ./proj2 input_file [FCFS|RR|SJF] [time_quantum]
  time_quantum (positive integer) is required for RR and is
  ignored by FCFS and SJF.
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$ ./proj2 missing.txt FCFS
Error: cannot open input file 'missing.txt'
jacobr@LAPTOP-P4Q45RUC:/mnt/c/Users/jake5/.vscode/C++ code/.vscode/operating_systems/assignment5$
```

Figure 10: Test case 4, invalid command lines and a missing input file

4. Summary of Results

Test	Algorithm	Quantum	Expected	Program output	Result
1	FCFS	-	9	9	Pass
1	SJF	-	7.75	7.75	Pass
1	RR	5	5.5	5.5	Pass
2	FCFS	-	0.25	0.25	Pass
2	RR	2	0.25	0.25	Pass
3	FCFS	-	6.6	6.6	Pass
3	SJF	-	3.6	3.6	Pass
3	RR	3	6.2	6.2	Pass
4	Error cases	-	Error message	Error message	Pass